# Nonlinear Function Approximation: Neural Networks

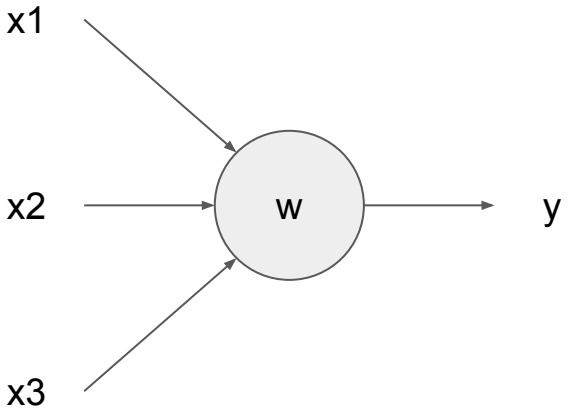
Genya Ishigaki

# Linear Approximation to Nonlinear

- A linear combination of state features is used to represent a value function
- Linear function approximation
  - Provides some nice theoretical guarantee on the learning convergence [Refer to the textbook]
  - But! Is not sufficient to represent a complex value function in practical tasks (underfitting)
- Nonlinear function approximation by neural networks (NNs)

# The Perceptron

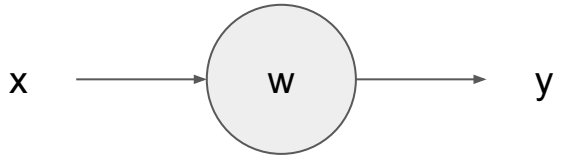
- A perceptron is an artificial neuron that takes a collection of binary inputs and produces a binary output
- The output of the perceptron is determined by summing up the weighted inputs and thresholding the result:
  - If the weighted sum is larger than the threshold *b*,
  - the output is one (and zero otherwise)




$$y = \begin{cases} 1 & \mathbf{w}^\top \mathbf{x} > b \\ 0 & \text{otherwise} \end{cases}$$

# An Example: NOT Perceptron

- Weight
  - `w = -1`
- Threshold
  - `b = -0.5`



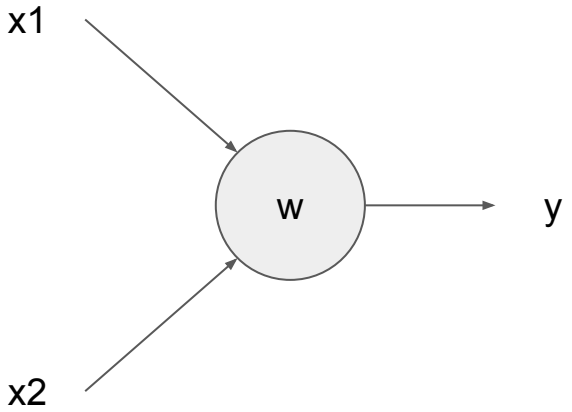

$$y = \begin{cases} 1 & -x > -0.5 \\ 0 & \text{otherwise} \end{cases}$$

# Milestone Questions: OR Perceptron

- Create OR Perceptron
- Create AND Perceptron

# An Example: OR Perceptron

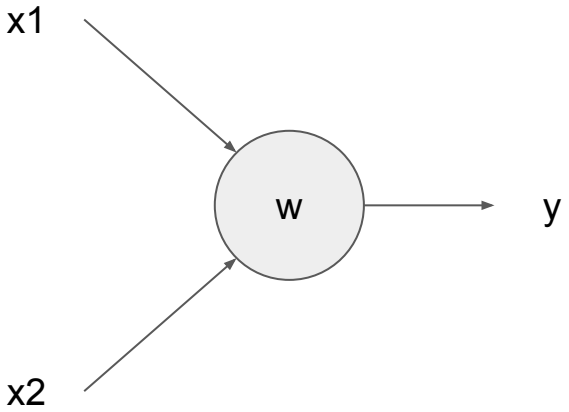
- Weight
  - `w1 = w2 = 1`
- Threshold
  - `b = 0`




$$y = \begin{cases} 1 & x_1 + x_2 > 0 \\ 0 & \text{otherwise} \end{cases}$$

# An Example: AND Perceptron

- Weight
  - `w1 = w2 = 1`
- Threshold
  - `b = 1.5`



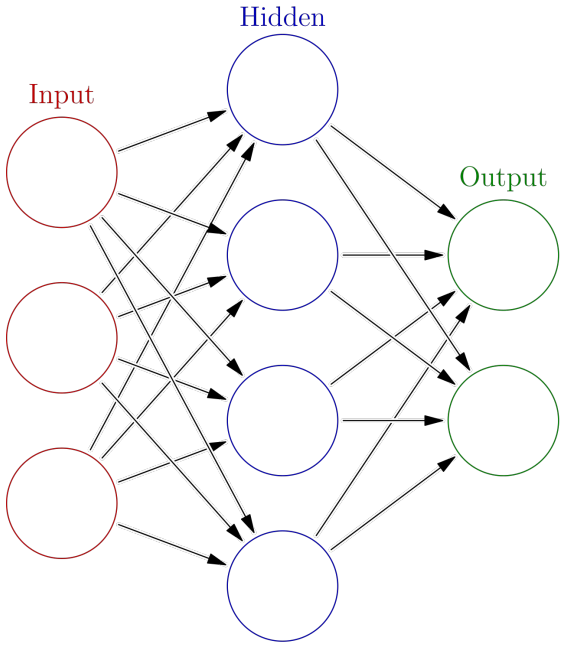

$$y = \begin{cases} 1 & x_1 + x_2 > 1.5 \\ 0 & \text{otherwise} \end{cases}$$

# Beyond Perceptrons: Neural Networks

- Feedforward neural networks to represent a more complex (nonlinear) functions
- There is no backward connections

$$f(\boldsymbol{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\boldsymbol{x})))$$

- Artificial neurons that have
  - Real number inputs/outputs and
  - (possibly) nonlinear activation functions
    1. The Sigmoid Neuron
    2. Rectified Linear Units (ReLU)
    3. Softmax

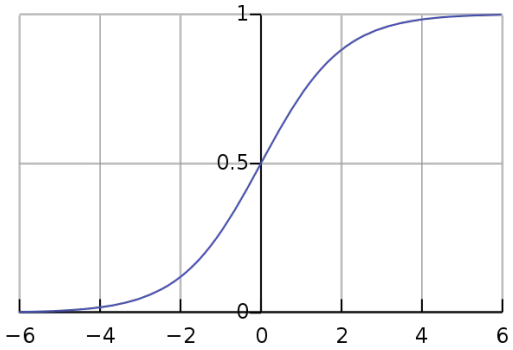


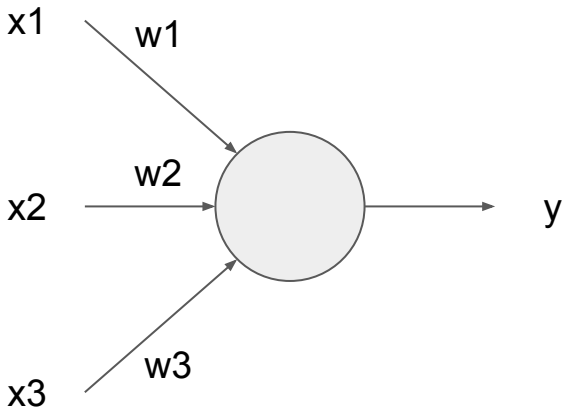

https://en.wikipedia.org/wiki/Artificial_neural_network

# 1. The Sigmoid Neuron

- Input: Real numbers
- Output: (0, 1)
- Differentiable!
- With one hidden layer with a sigmoid activation, the NN can learn a nonlinear function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



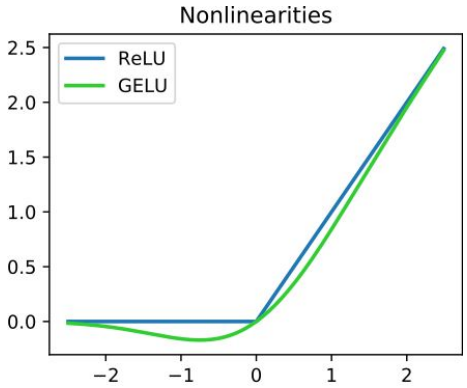



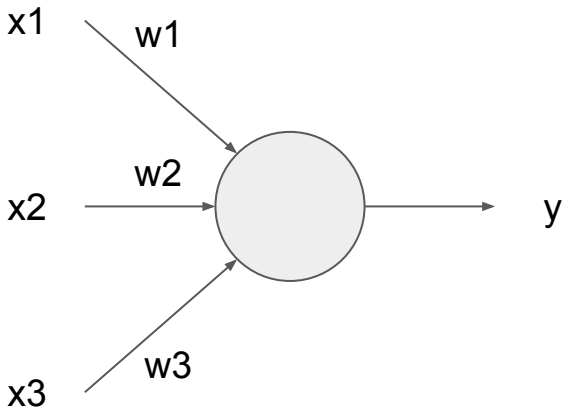

$$y = \sigma(w_1 x_1 + w_2 x_2 + w_3 x_3)$$

# 2. Rectified Linear Units (ReLU)

- Input: Real numbers
- Output:
  - The input if it is positive
  - Zero otherwise
- GELU (Gaussian-Error Linear Unit) is a smooth approximation of ReLU

$$\mathrm{ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{otherwise} \end{cases}$$



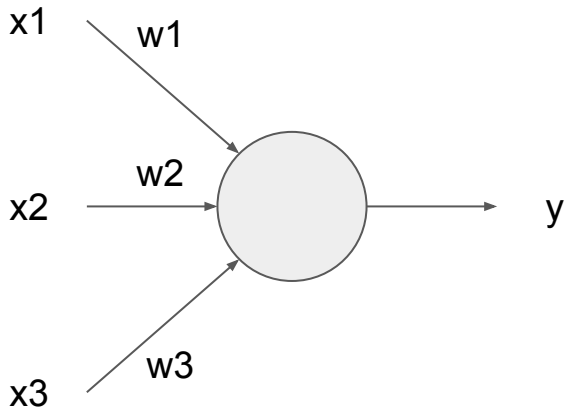




$$y = \mathrm{ReLU}(w_1 x_1 + w_2 x_2 + w_3 x_3)$$

# 3. Softmax

- Input: Real numbers
- Output: (0, 1)
- Conversion of a vector of K real numbers into a probability distribution of K possible outcomes

$$\text{softmax}(\mathbf{x})_i = \frac{e^{\mathbf{x}_i}}{\sum_{j=1}^{K} e^{\mathbf{x}_i}}$$

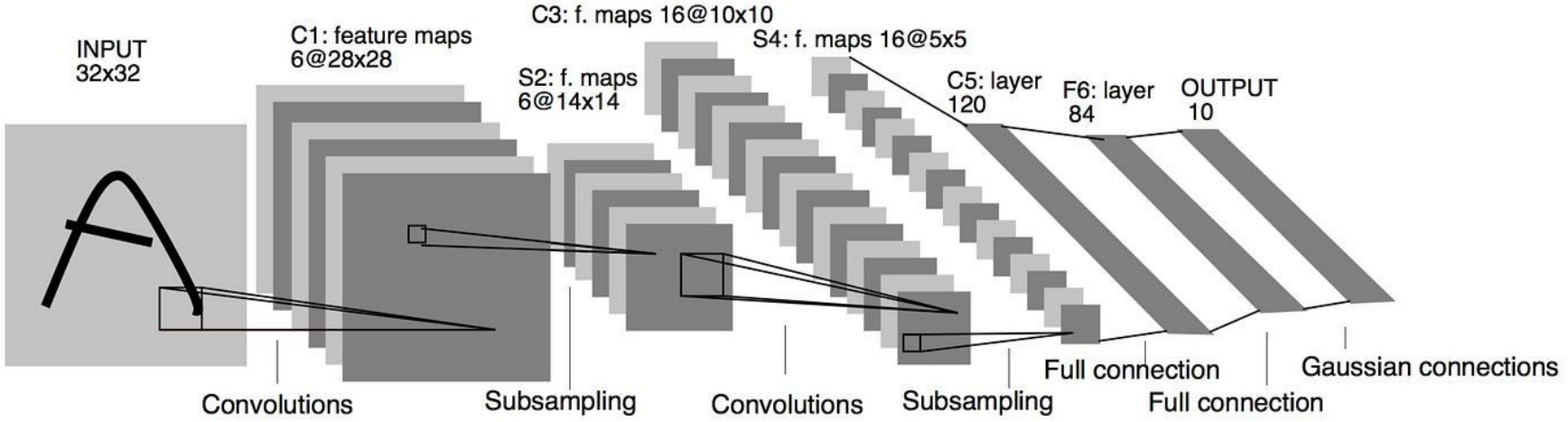



$$y = \text{softmax}(w_1 x_1 + w_2 x_2 + w_3 x_3)$$

# Universal Approximation Theorems

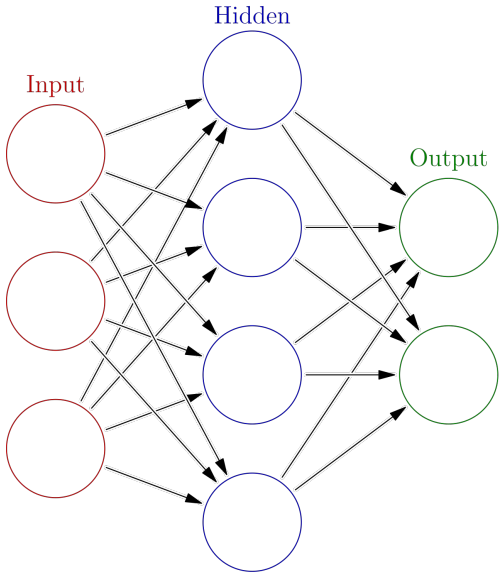
- A set of theorems that put limits on what neural networks (typically feedforward networks) can theoretically learn
- [Cybenko, 1989] An ANN with a single hidden layer containing a large enough finite number of sigmoid units can approximate any continuous function on a compact region of the network's input space to any degree of accuracy.
- However, both experience and theory show that approximating the complex functions needed for many tasks is made easier abstractions that are hierarchical compositions of many layers of lower-level abstractions
    - That is, abstractions produced by deep architectures such as ANNs with many hidden layers [Bengio, 2009]

# Abstractions produced by Deep Architectures



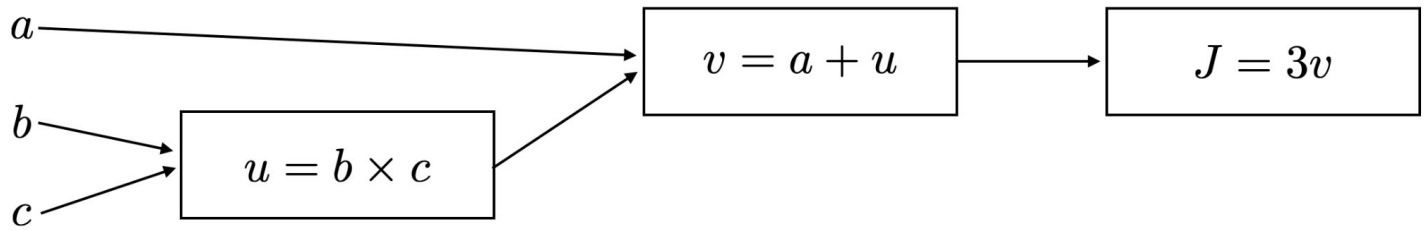
# Inference and Backprop

- Forward propagation (Inference): Given an input, we obtain the output
- Back propagation (Backprop): After obtaining the output, we flow back the loss information to compute the gradient (that are used to update the weight parameters)
  - Analytically, computing the gradient is not too difficult
  - However, numerically evaluating such expressions may be expensive
  - The backprop algorithm is a computationally efficient way to obtain the gradient values



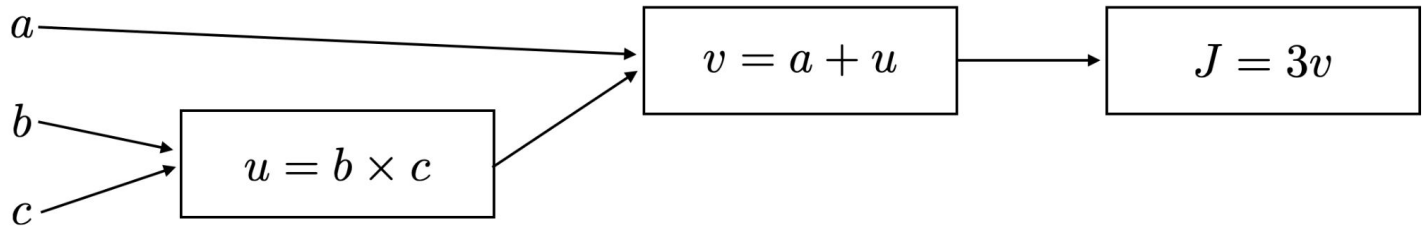
# Computational Graph

- A directed graph where the nodes correspond to mathematical operations
- A way of expressing and evaluating a mathematical expression
- J(a, b, c) = 3(a + bc)



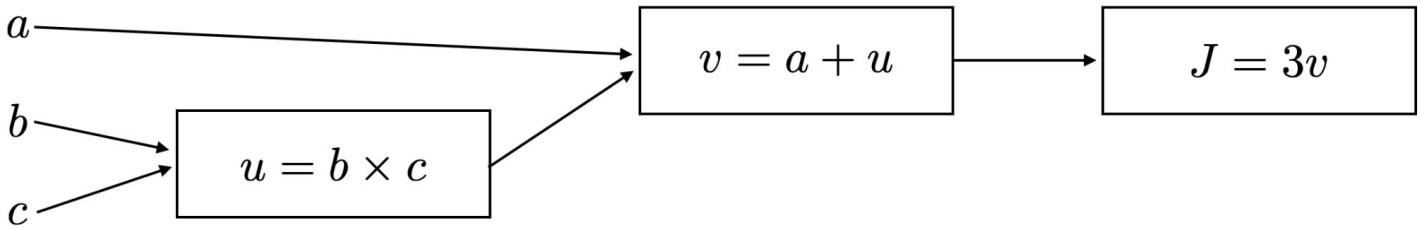
# Computational Graph

- J(a, b, c) = 3(a + bc)
- How does J change with a change of v?
  - v = 11 → v = 11.001
  - J = 33 → J = 33.003
  - J increases 3 times the increase of v: $\frac{dJ}{dv} = 3$ (matches with the analytical result)



# Computational Graph

- J(a, b, c) = 3(a + bc)
- How does J change with a change of a?

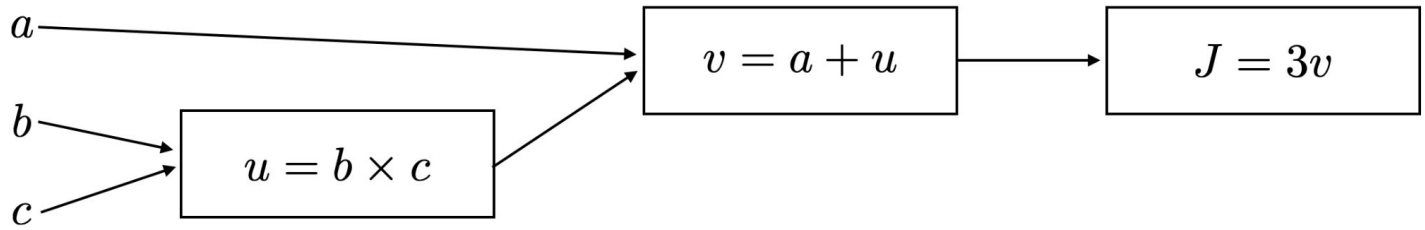


# Computational Graph

- J(a, b, c) = 3(a + bc)
- How does J change with a change of a?
  - a = 5 → a = 5.001
  - V = 11 → v = 11.001
  - J = 33 → J = 33.003
  - J increases 3 times the increase of a

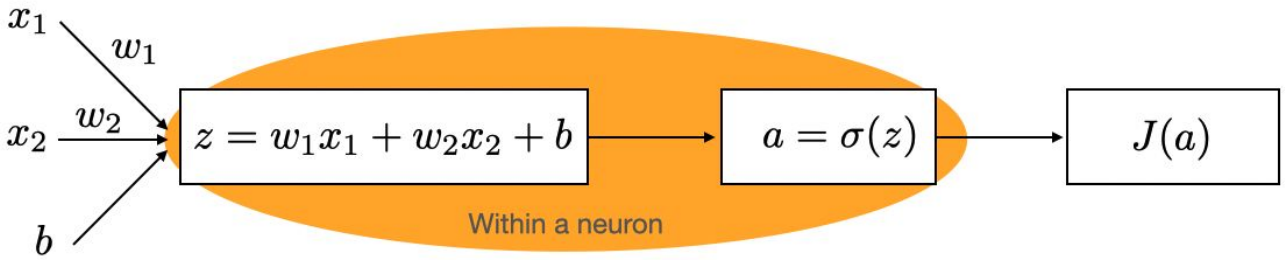
(Chain rule)

$$\frac{dJ}{da} = \frac{dJ}{dv}\frac{dv}{da} = 3 \times 1 = 3$$



# Updating the Parameters

- Given a loss value in the output layer of a NN
- Tune weights {w} and biases {b} to minimize the loss
- Repeat this for a lot of samples to learn the best parameters to map the inputs to the approximately best outputs

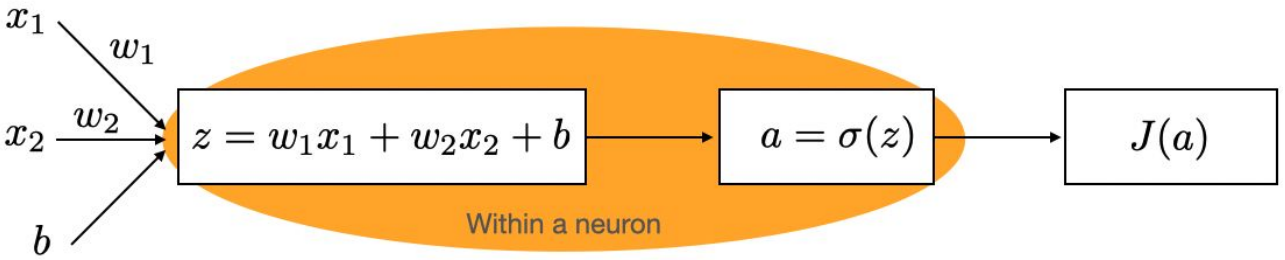


# Derivatives with a Computation Graph

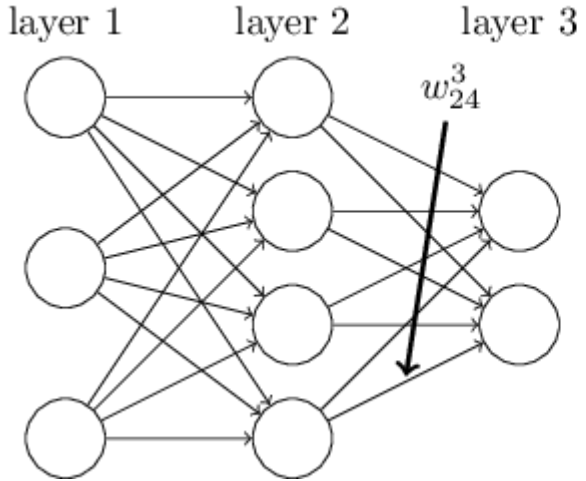
- The training dataset: $\left\{\left(\mathbf{x}^{(M)}, y^{(1)}\right), \ldots, \left(\mathbf{x}^{(M)}, y^{(M)}\right)\right\}$
- $a\left(\mathbf{x}^{(m)}, \mathbf{w}, b\right)$ is the output of the neural network for the m-th sample
- Assume J is a loss function we are minimizing

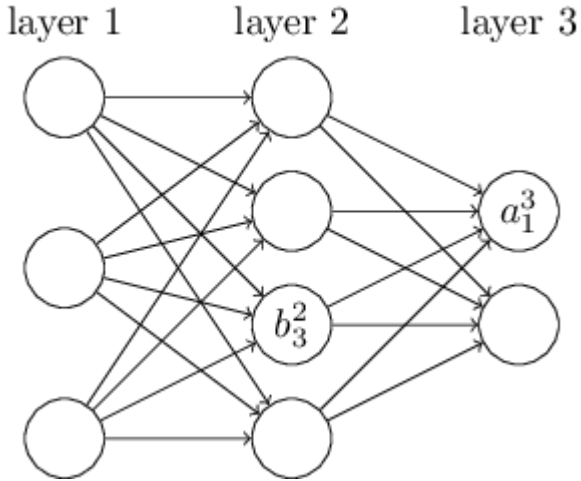
$$J(a) = \frac{1}{2M} \sum_{m} \left|\left| y^{(m)} - a\left(x^{(m)}, w, b\right) \right|\right|^2$$

- We want to update the parameters w and b so the loss is minimized



# How the Backprop Works: Notation




$w^l_{jk}$ is the weight from the $k^{\text{th}}$ neuron in the $(l-1)^{\text{th}}$ layer to the $j^{\text{th}}$ neuron in the $l^{\text{th}}$ layer

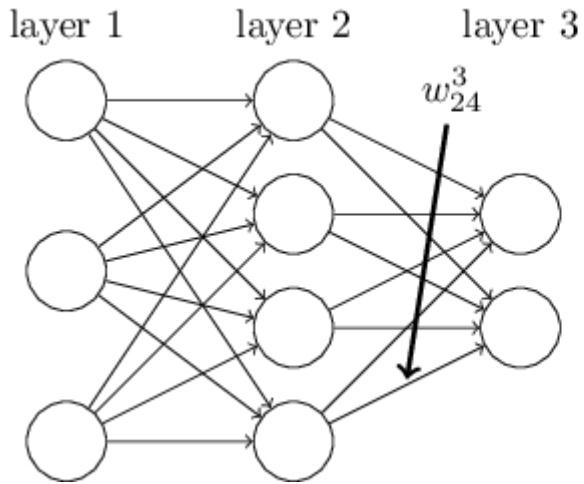
Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2015

# How the Backprop Works

- An output of a neuron can be represented with the weights, bias, and an activation function

$$a_j^l = \sigma\left(\sum_k w_{jk}^l a_k^{l-1} + b_j^l\right)$$

- Let's think about a concrete example, using $a_2^3$




$w_{jk}^l$ is the weight from the $k^{\text{th}}$ neuron in the $(l-1)^{\text{th}}$ layer to the $j^{\text{th}}$ neuron in the $l^{\text{th}}$ layer

Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2015

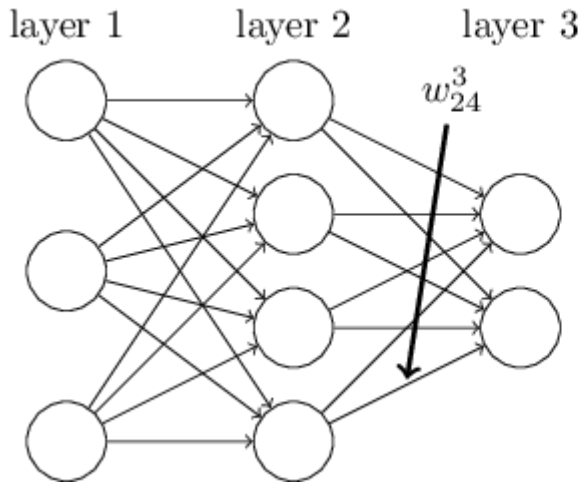
# How the Backprop Works

- An output of a neuron can be represented with the weights, bias, and an activation function

$$a_j^l = \sigma\left(\sum_k w_{jk}^l a_k^{l-1} + b_j^l\right)$$

- Stacking all neurons, we use a vectorized notation for each layer

$$a^l = \sigma\left(w^l a^{l-1} + b^l\right)$$



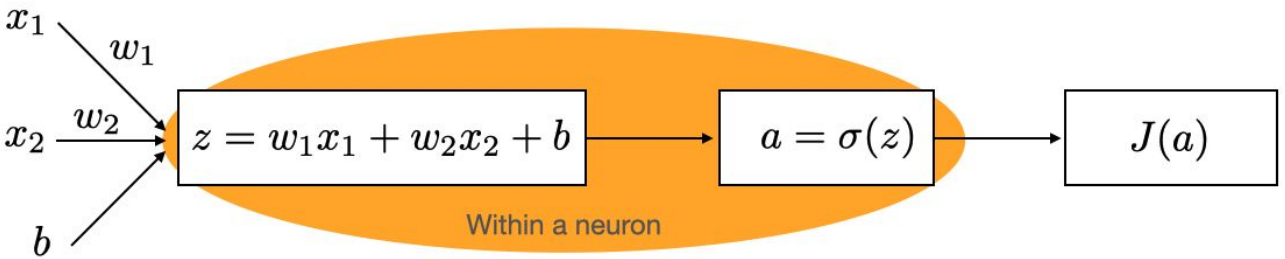

$w_{jk}^l$ is the weight from the $k^{\text{th}}$ neuron in the $(l-1)^{\text{th}}$ layer to the $j^{\text{th}}$ neuron in the $l^{\text{th}}$ layer

Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2015

# How the Backprop Works

- We are interested in “how sensitive each weight (or bias) is to the loss function”
  - e.g. Changing w1 to (w1 + d) reduces the loss. What is ideal d?
- That is the gradient of the loss function!
  - Using the chain rule, we compute the gradient to decide the d value

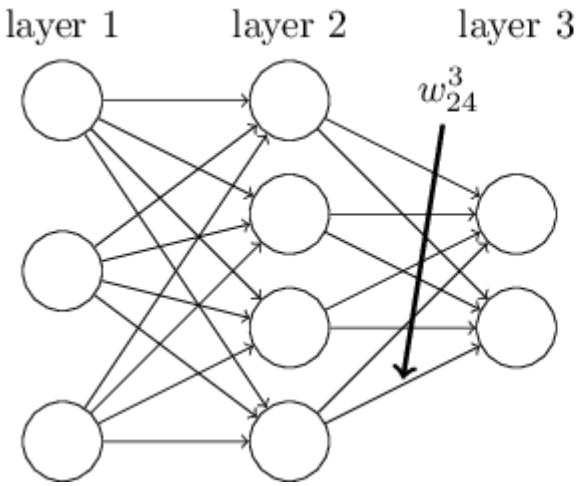
$$\frac{\partial J(a)}{\partial w_{jk}^l} = \frac{\partial J(a)}{\partial z_j^l}\frac{\partial z_j^l}{\partial w_{jk}^l} \qquad \frac{\partial J(a)}{\partial b_j^l} = \frac{\partial J(a)}{\partial z_j^l}\frac{\partial z_j^l}{\partial b_j^l}$$



# How the Backprop Works

- The loss function for a single data point

$$J(a) = \frac{1}{2}\left|\left|y^{(m)} - a\left(x^{(m)}, w, b\right)\right|\right|^2$$




- The derivative of the loss function in the last layer L is calculated as follows

$$\frac{\partial J}{\partial z_j^L} = -\left(y_j^{(m)} - a_j^L\right)\frac{\partial a_j^L}{\partial z_j^L}$$

$$= -\left(y_j^{(m)} - a_j^L\right)\frac{\partial \sigma\left(z_j^L\right)}{\partial z_j^L}$$

$$= -\left(y_j^{(m)} - a_j^L\right)\sigma\left(z_j^L\right)\left(1 - \sigma\left(z_j^L\right)\right)$$

$$\frac{d\sigma(z)}{z} = \sigma(z)(1 - \sigma(z))$$

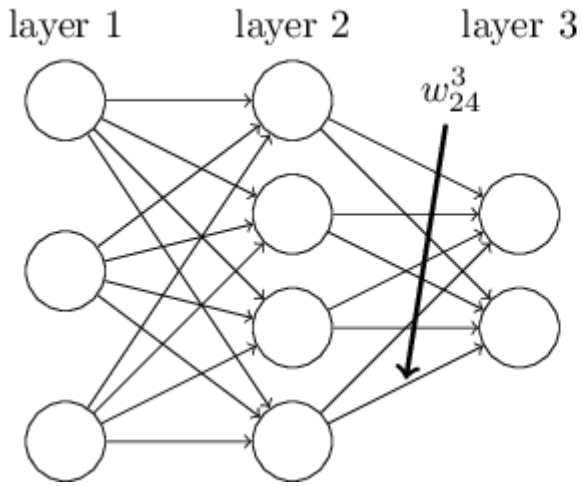
Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2015

# How the Backprop Works

- Remember:

$$z_j^l = \sum_k w_{jk}^l a_k^{l-1} + b_j^l$$

$$\frac{\partial z_j^l}{\partial w_{jk}^l} = a_k^{l-1}$$




- Therefore, the gradient we want to update the weight parameter is:

$$\frac{\partial J}{\partial w_{jk}^L} = \frac{\partial J}{\partial z_j^L}\frac{\partial z_j^L}{\partial w_{jk}^L} = -\left(y_j^{(m)} - a_j^L\right)\sigma\left(z_j^L\right)\left(1-\sigma\left(z_j^L\right)\right)a_j^{L-1}$$

Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2015

# The Backprop

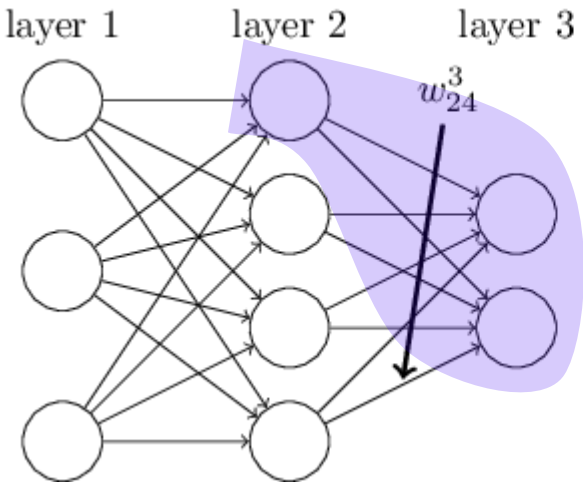
- Since we are solving the minimization problem of the loss function
- We should move the parameters to the negative direction of the gradient

$$w_{jk}^{l} = w_{jk}^{l} - \alpha \frac{\partial J}{\partial w_{jk}^{l}}$$

- We need to compute the gradient in each layer
  - But the chain rule helps us to **back**-propagate the loss

$$\frac{\partial J}{\partial z_j^l} = \sum_k \frac{\partial J}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l}$$

This term is already computed in the previous layer!



# The Backprop

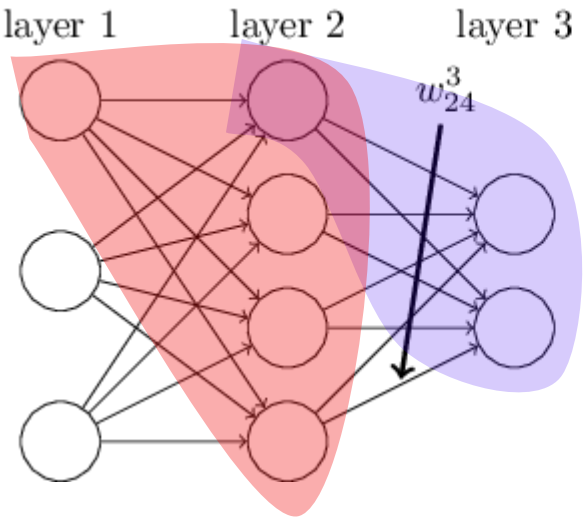
- Since we are solving the minimization problem of the loss function
- We should move the parameters to the negative direction of the gradient

$$w_{jk}^{l} = w_{jk}^{l} - \alpha \frac{\partial J}{\partial w_{jk}^{l}}$$

- We need to compute the gradient in each layer
  - But the chain rule helps us to **back**-propagate the loss

$$\frac{\partial J}{\partial z_j^l} = \sum_k \frac{\partial J}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l}$$

This term is already computed in the previous layer!



# The Backprop

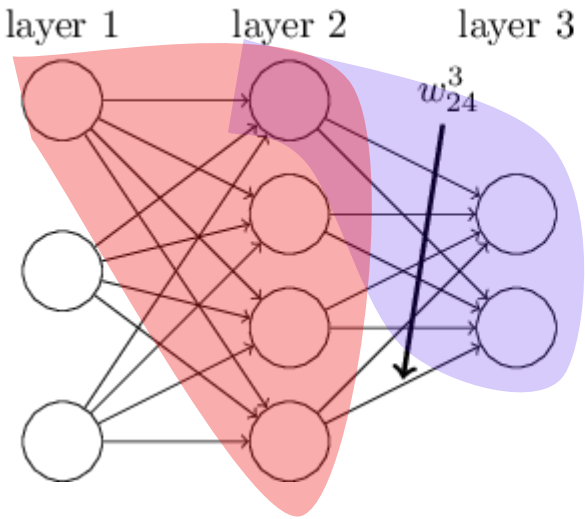
- Remember:

$$z_k^{l+1} = \sum_j w_{kj}^{l+1} a_j^l + b_k^{l+1} = \sum_j w_{kj}^{l+1} \sigma\left(z_j^l\right) + b_k^{l+1}$$

- Therefore,

$$\frac{\partial z_k^{l+1}}{\partial z_j^l} = w_{kj}^{l+1} \sigma'(z_j^l)$$

$$\frac{\partial J}{\partial z_j^l} = \sum_k \frac{\partial J}{\partial z_k^{l+1}} w_{kj}^{l+1} \sigma'(z_j^l)$$



# Reference

- Backpropagation calculus
  - https://www.youtube.com/watch?v=tIeHLnjs5U8
- Neural Networks and Deep Learning
  - http://neuralnetworksanddeeplearning.com/